INDEX

INDEX

CHAPTER 1: INTRODUCTION AND OBJECTIVE

1.1 Introduction:

Education plays a vital role in the development of society, and schools handle a large amount of information every day. Student admissions, teacher records, attendance, examination results, fee collection, notifications, complaints, and leave applications are some of the important activities performed regularly in every educational institution. In many schools, these tasks are still managed manually using registers, notebooks, and paper files. Although this traditional method has been followed for many years, it is time-consuming, difficult to maintain, and more likely to produce errors.

With the advancement of computer technology, schools are increasingly adopting computerized systems to improve their administrative work. A School Management System provides a centralized platform that stores and manages all important information digitally. It allows authorized users to perform different tasks quickly while ensuring that records remain organized and easily accessible.

The **School Management System** developed in this project is a desktop application created using the **C++ programming language**. The project mainly focuses on applying Object-Oriented Programming (OOP) concepts together with file handling to build a practical real-world application. The system provides different login panels for **Super Admin, Principal, Teacher, Student, and Parent**, where each user has different responsibilities and permissions according to their role.

The Super Admin is responsible for managing schools and principals. The Principal manages teachers and oversees school activities. Teachers can admit students, mark attendance, enter examination results, set fees, send notifications, and request student removal. Students can access their academic information, while parents can view information related to their children. This role-based structure improves security and ensures that users can only access the features assigned to them.

The application stores all information permanently using text files, allowing data to remain available even after the program is closed. The use of file handling also demonstrates practical implementation of C++ concepts such as classes, objects, functions, inheritance, encapsulation, and data management.

Overall, the School Management System simplifies school administration, reduces paperwork, improves data accuracy, saves valuable time, and provides a user-friendly solution for managing school records efficiently.

1.2 Project Profile:

The School Management System is a menu-driven desktop application designed to automate various administrative and academic activities performed in a school. The project has been developed using the C++ programming language and follows the principles of Object-Oriented Programming to make the application modular, organized, and easy to maintain.

The application provides separate login accounts for different categories of users, ensuring proper access control and data security. Each user performs only those operations that belong to their responsibilities. Data is stored using text files through file handling techniques, allowing records to be saved permanently without requiring any external database.

The project has been developed primarily for educational purposes to demonstrate the practical implementation of programming concepts learned during the course. It also provides experience in designing menu-driven applications, implementing multiple user roles, validating user input, generating unique IDs, and maintaining structured records

Project Details

- **Project Title:** School Management System
- **Project Type:** Desktop Application
- **Category:** Educational Management System
- **Programming Language:** C++
- **Programming Paradigm:** Object-Oriented Programming (OOP)
- **Development Environment:** Visual Studio Code
- **Compiler:** GNU G++ / MinGW
- **Operating System:** Windows
- **Database:** File Handling (.txt files)
- **Interface:** Console-Based Menu Driven System

1.3 Objectives:

- To reduce manual paperwork involved in school administration.
- To provide role-based access for Super Admin, Principal, Teacher, Student, and Parent.
- To maintain school information in an organized digital format.
- To generate unique IDs automatically for different users.
- To manage attendance records efficiently.
- To maintain examination results accurately.
- To store fee-related information securely.
- To provide notification and complaint management facilities.
- To minimize human errors during record management.

1.4 Existing Manual System:

In the traditional school management system, almost every activity is performed manually using registers, notebooks, files, and paperwork. Student admission forms, teacher records, attendance registers, examination results, fee receipts, and other documents are maintained physically by school staff. Although this approach is simple, it becomes increasingly difficult to manage as the number of students and employees grows.

Searching for a particular student's information often requires checking multiple registers, which consumes a considerable amount of time. Updating records manually may also result in overwriting, duplication, or missing information. Generating reports becomes difficult because every calculation must be performed manually.

Manual systems also increase the possibility of losing important documents due to physical damage, misplacement, or improper storage. Maintaining consistency among different records becomes challenging, especially when multiple staff members update information simultaneously.

Therefore, many educational institutions are replacing manual systems with computerized solutions that improve efficiency, security, and data accuracy.

1.4.1 Limitations of the Existing Manual System:

- Requires large amounts of paperwork.
- Time-consuming for maintaining records.
- Difficult to search old information.
- Duplicate records may occur.
- Records can be lost or damaged easily.
- Updating information requires significant manual effort.
- Report generation becomes difficult.
- Data consistency cannot always be maintained.
- Security of confidential information is limited.
- Managing large numbers of students becomes complicated.

1.5 Proposed System:

The proposed School Management System is designed to replace manual record management with a computerized solution developed using C++. The application automates several school activities by storing information digitally through file handling.

Different users receive separate login credentials according to their roles. The Super Admin manages schools and principals, Principals manage teachers, Teachers manage students and academic activities, Students access their information, and Parents monitor their children's records.

The system automatically generates unique IDs, validates user input, stores records permanently, and provides search, update, and delete operations. Since all information is maintained digitally, retrieving records becomes faster and more accurate.

The proposed system reduces paperwork, minimizes errors, improves security, and provides an organized method for maintaining school information efficiently.

1.5.1 Advantages of Proposed System:

- Reduces paperwork significantly.
- Saves time during daily operations.
- Improves data accuracy.
- Stores records permanently using file handling.
- Easy searching and retrieval of records.
- Faster updating and deletion of information.
- Provides role-based security.
- Generates unique IDs automatically.
- User-friendly menu-driven interface.
- Better organization of school records.
- Reduces duplicate entries.
- Easy maintenance and future enhancement.
- Demonstrates practical implementation of C++ concepts.

CHAPTER 2: REQUIREMENT ANALYSIS

2.1 Introduction:

Requirement Analysis is one of the most important stages of software development. During this phase, the functional and non-functional requirements of the project are identified and analyzed before implementation begins. Proper requirement analysis helps developers understand the resources, software, hardware, and user requirements needed for the successful completion of the project.

The School Management System has been designed to satisfy the requirements of educational institutions by providing an easy-to-use, secure, and efficient application for managing different school activities.

2.2 Software Requirements:

- ✓ **Operating System:** Windows.
- ✓ **Programming Language:** C++.
- ✓ **Code Editor:** Visual Studio Code.
- ✓ **Compiler:** C++ Compiler.
- ✓ **Storage:** Local files for storing records.

2.3 Hardware Requirement:

- ✓ Minimum 4GB RAM
- ✓ Dual-core processor.
- ✓ Compatible with desktops or laptops.

2.4 Training Requirement:

Basic understanding of:

- Computer operation.
- Menu-based applications.
- Different user roles and their functions.
- Basic C++ knowledge for maintaining the program.

2.5 Descriptions:

The School Management System consists of multiple modules that work together to perform different administrative and academic operations.

- **Super Admin:** Creates schools, manages principals, searches schools, edits school information, and removes records.
- **Principal:** Manages teachers belonging to the assigned school and monitors school-related activities.
- **Teacher:** Admits students, records attendance, enters examination results, updates marks, manages fees, sends notifications, and submits removal requests.
- **Student:** Views attendance, examination results, fee details, notifications, and personal information.
- **Parent:** Accesses information related to their child, including attendance, results, notifications, and fee records.

The interaction among these modules provides an organized management system for the entire school.

2.6 Why C++?

C++ has been selected because it is one of the most powerful programming languages for learning Object-Oriented Programming and system development.

Advantages include:

- Supports Object-Oriented Programming.
- High execution speed.
- Efficient memory management.
- Powerful file handling capabilities.
- Supports classes and inheritance.
- Suitable for menu-driven applications.
- Easy to implement modular programming.
- Widely used in academic projects.
- Helps strengthen programming fundamentals.

CHAPTER 3: FEASIBILITY STUDY

3.1 Introduction:

A feasibility study is performed before developing a software project to determine whether the project is practical, economical, and beneficial. It evaluates different factors such as technical resources, financial requirements, operational efficiency, and implementation possibilities.

For the School Management System, the feasibility study confirms that the project can be successfully developed using available software and hardware resources without significant cost.

3.2 Technical Feasibility:

Technical feasibility determines whether the required technology, hardware, and software are available for developing the application.

The School Management System is technically feasible because it uses C++, a standard programming language supported by most operating systems and compilers. The project requires only a normal computer, Visual Studio Code, and a C++ compiler. File handling eliminates the need for an external database, making development simpler while still providing permanent storage.

The application is lightweight, requires minimal system resources, and can run efficiently on ordinary desktop and laptop computers.

3.3 Economical Feasibility:

Economic feasibility determines whether the project can be completed within an affordable budget.

The School Management System requires no expensive software licenses or hardware. The compiler, development environment, and required programming tools are freely available. Since the project uses text files instead of commercial database software, the overall development cost remains very low.

Maintenance costs are also minimal because the application can be updated without additional licensing expenses.

3.4 Operational Feasibility:

Operational feasibility evaluates whether users can operate the application successfully.

The School Management System has been developed using a simple menu-driven interface that is easy to understand. Different users receive separate login panels according to their responsibilities, reducing confusion and improving security.

Since the application automates several manual activities, school staff can perform their daily work more efficiently. The system reduces paperwork, minimizes errors, and allows faster retrieval of information.

3.5 Conclusion:

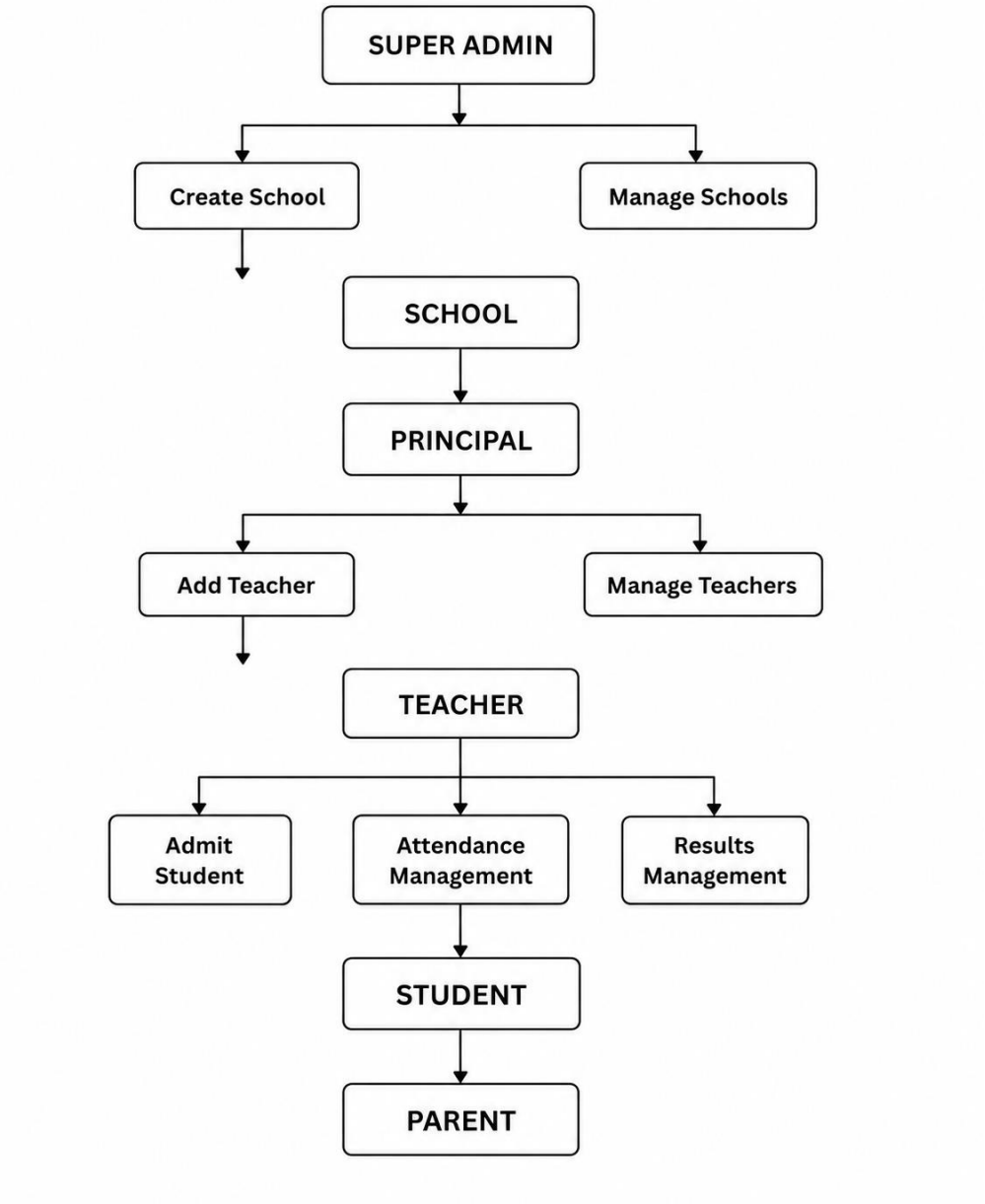
The feasibility study indicates that the School Management System is technically, economically, and operationally feasible. The project can be developed using available resources without high costs while providing significant improvements over the traditional manual system.

The system offers faster data management, improved accuracy, enhanced security, organized record keeping, and better overall efficiency. Therefore, implementing this computerized School Management System is a practical and beneficial solution for educational institutions.

Paradox

SCHOOL MANAGEMENT SYSTEM

BLUEPRINT :



CHAPTER 4: CODING AND OUTPUT

4.1 Coding:

```
#include <iostream>
#include <fstream>
#include <string>
#include <sstream>
#include <vector>
#include <iomanip>
#include <limits>
#include <cstdio>
#include <cctype>
#include <algorithm>
using namespace std;
namespace SMS {
const string BACK = "back";
const string EXIT = "exit";
void cls() {
#ifdef _WIN32
   system("cls");
#else
   system("clear");
#endif
}
void pauseScreen() {
   cout << "\nPress Enter to continue...";
   cin.ignore(numeric_limits<streamsize>::max(), '\n');
}
string lowerCopy(string s) {
   transform(s.begin(), s.end(), s.begin(),
         [](unsigned char c) { return static_cast<char>(tolower(c)); });
```

```
    return s;
}
bool isBack(const string& s) {
    return lowerCopy(s) == BACK;
}
bool isExit(const string& s) {
    return lowerCopy(s) == EXIT;
}
bool inputText(const string& prompt, string& value, bool allowEmpty = false) {
    while (true) {
        cout << prompt;
        getline(cin >> ws, value);
        if (isBack(value)) return false;
        if (isExit(value)) return false;
        if (!allowEmpty && value.empty()) {
            cout << "Invalid input\n";
            continue;
        }
        return true;
    }
}
bool inputInt(const string& prompt, int& value, int minValue = numeric_limits<int>::min(),
              int maxValue = numeric_limits<int>::max()) {
    while (true) {
        cout << prompt;
        string input;
        getline(cin >> ws, input);
        if (isBack(input) || isExit(input)) return false;
        stringstream ss(input);
        char extra;
        if (!(ss >> value) || (ss >> extra) || value < minValue || value > maxValue) {
```

```
            cout << "Invalid option\n";
            continue;
        }
        return true;
    }
}
bool inputDouble(const string& prompt, double& value, double minValue = 0) {
    while (true) {
        cout << prompt;
        string input;
        getline(cin >> ws, input);
        if (isBack(input) || isExit(input)) return false;
        stringstream ss(input);
        char extra;
        if (!(ss >> value) || (ss >> extra) || value < minValue) {
            cout << "Invalid option\n";
            continue;
        }
        return true;
    }
}
bool inputChar(const string& prompt, char& value) {
    while (true) {
        cout << prompt;
        string input;
        getline(cin >> ws, input);
        if (isBack(input) || isExit(input)) return false;
        if (input.size() != 1) {
            cout << "Invalid option\n";
            continue;
        }
```

```
        value = input[0];
        return true;
    }
}
void invalidChoice() {
    cout << "\nInvalid option\n";
    pauseScreen();
}
bool menuChoice(const string& prompt, int& choice, int minChoice, int maxChoice) {
    while (true) {
        cout << prompt;
        string input;
        getline(cin >> ws, input);
        if (isBack(input) || isExit(input)) return false;
        stringstream ss(input);
        char extra;
        if (!(ss >> choice) || (ss >> extra) || choice < minChoice || choice > maxChoice) {
            invalidChoice();
            cls();
            return true;
        }
        return true;
    }
}
class FileManager {
public:
    static string nextID(const string& fileName, const string& prefix) {
        ifstream file(fileName);
        string line;
        int maxNumber = 0;
        while (getline(file, line)) {
```

```
        size_t pos = line.find('|');
        string id = (pos == string::npos) ? line : line.substr(0, pos);
        if (id.rfind(prefix, 0) == 0 && id.size() > prefix.size()) {
            string number = id.substr(prefix.size());
            bool numeric = !number.empty() &&
                all_of(number.begin(), number.end(),
                    [](unsigned char c) { return isdigit(c); });
            if (numeric) {
                try {
                    maxNumber = max(maxNumber, stoi(number));
                } catch (...) {}
            }
        }
    }
    ostringstream out;
    out << prefix << setw(3) << setfill('0') << (maxNumber + 1);
    return out.str();
}
static bool idExists(const string& fileName, const string& id) {
    ifstream file(fileName);
    string line;
    while (getline(file, line)) {
        size_t pos = line.find('|');
        string stored = (pos == string::npos) ? line : line.substr(0, pos);
        if (stored == id) return true;
    }
    return false;
}
static bool findByID(const string& fileName, const string& id, string& lineOut) {
    ifstream file(fileName);
    string line;
```

```
    while (getline(file, line)) {
        size_t pos = line.find('|');
        string stored = (pos == string::npos) ? line : line.substr(0, pos);
        if (stored == id) {
            lineOut = line;
            return true;
        }
    }
    return false;
}
static vector<string> readAll(const string& fileName) {
    vector<string> lines;
    ifstream file(fileName);
    string line;
    while (getline(file, line)) {
        if (!line.empty()) lines.push_back(line);
    }
    return lines;
}
static bool append(const string& fileName, const string& data) {
    ofstream file(fileName, ios::app);
    if (!file.is_open()) return false;
    file << data << '\n';
    return true;
}
static bool removeByID(const string& fileName, const string& id) {
    ifstream file(fileName);
    ofstream temp("temp_sms.txt");
    if (!file.is_open() || !temp.is_open()) return false;
    bool removed = false;
    string line;
```

```
    while (getline(file, line)) {
        size_t pos = line.find('|');
        string stored = (pos == string::npos) ? line : line.substr(0, pos);
        if (stored == id) {
            removed = true;
        } else {
            temp << line << '\n';
        }
    }
    file.close();
    temp.close();
    remove(fileName.c_str());
    rename("temp_sms.txt", fileName.c_str());
    return removed;
}
static bool removeSchoolByID(const string& schoolID) {
    return removeByID("schools.txt", schoolID);
}
static bool updateLine(const string& fileName, const string& matchID,
                       const string& newLine) {
    ifstream file(fileName);
    ofstream temp("temp_sms.txt");
    if (!file.is_open() || !temp.is_open()) return false;
    bool updated = false;
    string line;
    while (getline(file, line)) {
        size_t pos = line.find('|');
        string stored = (pos == string::npos) ? line : line.substr(0, pos);
        if (stored == matchID) {
            temp << newLine << '\n';
            updated = true;
```

```
        } else {
            temp << line << '\n';
        }
    }
    file.close();
    temp.close();
    remove(fileName.c_str());
    rename("temp_sms.txt", fileName.c_str());
    return updated;
  }
  static bool saveSchool(const string& data) { return append("schools.txt", data); }
  static bool savePrincipal(const string& data) { return append("principals.txt", data); }
  static bool saveTeacher(const string& data) { return append("teachers.txt", data); }
  static bool saveStudent(const string& data) { return append("students.txt", data); }
  static bool saveParent(const string& data) { return append("parents.txt", data); }
  static bool saveAttendance(const string& data) { return append("attendance.txt", data); }
  static bool saveResult(const string& data) { return append("results.txt", data); }
  static bool saveFees(const string& data) { return append("fees.txt", data); }
  static bool saveNotification(const string& data) { return append("notifications.txt", data);
}
  static bool saveComplaint(const string& data) { return append("complaints.txt", data); }
  static bool saveLeaveRequest(const string& data) { return append("leave_requests.txt",
data); }
  static bool saveRemovalRequest(const string& data) { return
append("removal_requests.txt", data); }
  static void load(const string& fileName) {
    vector<string> lines = readAll(fileName);
    if (lines.empty()) {
        cout << "No records found.\n";
        return;
    }
    for (const string& line : lines) cout << line << '\n';
```

```
}
static void loadSchools() { load("schools.txt"); }
static void loadPrincipals() { load("principals.txt"); }
static void loadTeachers() { load("teachers.txt"); }
static void loadStudents() { load("students.txt"); }
static void loadAttendance() { load("attendance.txt"); }
static void loadResults() { load("results.txt"); }
static void loadFees() { load("fees.txt"); }
static void loadNotifications() { load("notifications.txt"); }
static void loadComplaints() { load("complaints.txt"); }
static void loadLeaveRequests() { load("leave_requests.txt"); }
static void loadRemovalRequests() { load("removal_requests.txt"); }
static bool verifyLogin(const string& fileName, const string& id, const string& password) {
    string line;
    if (!findByID(fileName, id, line)) return false;
    vector<string> fields = split(line);
    return fields.size() >= 2 && fields[0] == id && fields[1] == password;
}
static bool verifyStudentLogin(const string& id, const string& password) {
    return verifyLogin("students.txt", id, password);
}
static bool verifyPrincipalLogin(const string& id, const string& password) {
    return verifyLogin("principals.txt", id, password);
}
static bool verifyTeacherLogin(const string& id, const string& password) {
    return verifyLogin("teachers.txt", id, password);
}
static bool verifyParentLogin(const string& id, const string& password) {
    return verifyLogin("parents.txt", id, password);
}
static vector<string> split(const string& line) {
```

```
    vector<string> fields;
    string field;
    stringstream ss(line);
    while (getline(ss, field, '|')) fields.push_back(field);
    return fields;
}
static string getField(const string& fileName, const string& id, size_t index) {
    string line;
    if (!findByID(fileName, id, line)) return "";
    vector<string> fields = split(line);
    if (index >= fields.size()) return "";
    return fields[index];
}
static string getSchoolName(const string& schoolID) {
    string name = getField("schools.txt", schoolID, 1);
    return name.empty() ? "Unknown School" : name;
}
static bool schoolExists(const string& schoolID) {
    return idExists("schools.txt", schoolID);
}
static bool principalExists(const string& principalID) {
    return idExists("principals.txt", principalID);
}
static bool teacherExists(const string& teacherID) {
    return idExists("teachers.txt", teacherID);
}
static bool studentExists(const string& studentID) {
    return idExists("students.txt", studentID);
}
static string getTeacherSchoolID(const string& teacherID) {
    return getField("teachers.txt", teacherID, 6);
```

```
}
static string getPrincipalSchoolID(const string& principalID) {
    return getField("principals.txt", principalID, 6);
}
static string getStudentSchoolID(const string& studentID) {
    return getField("students.txt", studentID, 6);
}
static string getStudentName(const string& studentID) {
    return getField("students.txt", studentID, 2);
}
static bool studentBelongsToSchool(const string& studentID, const string& schoolID) {
    return studentExists(studentID) && getStudentSchoolID(studentID) == schoolID;
}
static void showStudentProfile(const string& id) {
    string line;
    if (!findByID("students.txt", id, line)) {
        cout << "Student Not Found!\n";
        return;
    }
    vector<string> f = split(line);
    cout << "\nID: " << (f.size() > 0 ? f[0] : "") << '\n';
    cout << "Name: " << (f.size() > 2 ? f[2] : "") << '\n';
    cout << "DOB: " << (f.size() > 3 ? f[3] : "") << '\n';
    cout << "Phone: " << (f.size() > 5 ? f[5] : "") << '\n';
    cout << "School ID: " << (f.size() > 6 ? f[6] : "") << '\n';
    cout << "School: " << (f.size() > 6 ? getSchoolName(f[6]) : "") << '\n';
    cout << "Class: " << (f.size() > 7 ? f[7] : "") << '\n';
    cout << "Section: " << (f.size() > 8 ? f[8] : "") << '\n';
    cout << "Roll No: " << (f.size() > 9 ? f[9] : "") << '\n';
    cout << "Parent: " << (f.size() > 10 ? f[10] : "") << '\n';
}
```

```
static bool searchStudent(const string& id, const string& schoolID = "") {
    string line;
    if (!findByID("students.txt", id, line)) return false;
    vector<string> f = split(line);
    if (!schoolID.empty() && (f.size() <= 6 || f[6] != schoolID)) return false;
    cout << line << '\n';
    return true;
}
static bool searchTeacher(const string& id, const string& schoolID = "") {
    string line;
    if (!findByID("teachers.txt", id, line)) return false;
    vector<string> f = split(line);
    if (!schoolID.empty() && (f.size() <= 6 || f[6] != schoolID)) return false;
    cout << line << '\n';
    return true;
}
static bool updateResult(const string& studentID, const string& subject, int newMarks) {
    ifstream file("results.txt");
    ofstream temp("temp_sms.txt");
    if (!file.is_open() || !temp.is_open()) return false;
    bool updated = false;
    string line;
    while (getline(file, line)) {
        vector<string> f = split(line);
        if (f.size() >= 3 && f[0] == studentID && f[1] == subject) {
            temp << studentID << '|' << subject << '|' << newMarks << '\n';
            updated = true;
        } else {
            temp << line << '\n';
        }
    }
```

```
        file.close();
        temp.close();
        remove("results.txt");
        rename("temp_sms.txt", "results.txt");
        return updated;
    }
    static bool deleteSchool(const string& schoolID) {
        return removeSchoolByID(schoolID);
    }
};
class User {
protected:
    string id;
    string password;
    string name;
    string dob;
    string aadhar;
    string phone;
public:
    User() = default;
    void setBasicInfo(const string& id, const string& password, const string& name,
                      const string& dob, const string& aadhar, const string& phone) {
        this->id = id;
        this->password = password;
        this->name = name;
        this->dob = dob;
        this->aadhar = aadhar;
        this->phone = phone;
    }
    string getID() const { return id; }
    string getPassword() const { return password; }
```

```
    string getName() const { return name; }
    string getDOB() const { return dob; }
    string getAadhar() const { return aadhar; }
    string getPhone() const { return phone; }
    virtual void display() const {
        cout << "\nID: " << id << '\n';
        cout << "Name: " << name << '\n';
        cout << "DOB: " << dob << '\n';
        cout << "Phone: " << phone << '\n';
    }
    virtual ~User() = default;
};
class Parent : public User {
    string studentID;
    string studentName;
public:
    bool login(const string& loginID, const string& pass) {
        return FileManager::verifyParentLogin(loginID, pass);
    }
    void displayMenu() {
        while (true) {
            cls();
            cout << "===== PARENT PANEL =====\n\n";
            cout << "1. View Student Profile\n";
            cout << "2. View Attendance\n";
            cout << "3. View Results\n";
            cout << "4. View Fees\n";
            cout << "5. View Notifications\n";
            cout << "6. Submit Complaint\n";
            cout << "7. Leave School Request\n";
            cout << "8. Logout\n";
```

```
        int choice;
        if (!menuChoice("\nEnter Choice: ", choice, 1, 8)) return;
        switch (choice) {
            case 1: viewStudentProfile(); break;
            case 2: viewAttendance(); break;
            case 3: viewResults(); break;
            case 4: viewFees(); break;
            case 5: viewNotifications(); break;
            case 6: submitComplaint(); break;
            case 7: leaveSchoolRequest(); break;
            case 8: return;
        }
      }
   }
private:
   void viewStudentProfile() {
      cls();
      string id;
      if (!inputText("Enter Student ID (or back): ", id)) return;
      cout << "\n===== STUDENT PROFILE =====\n";
      FileManager::showStudentProfile(id);
      pauseScreen();
   }
   void viewAttendance() {
      cls();
      cout << "===== ATTENDANCE =====\n\n";
      FileManager::loadAttendance();
      pauseScreen();
   }
   void viewResults() {
      cls();
```

```
    cout << "===== RESULTS =====\n\n";
    FileManager::loadResults();
    pauseScreen();
}
void viewFees() {
    cls();
    cout << "===== FEES =====\n\n";
    FileManager::loadFees();
    pauseScreen();
}
void viewNotifications() {
    cls();
    cout << "===== NOTIFICATIONS =====\n\n";
    FileManager::loadNotifications();
    pauseScreen();
}
void submitComplaint() {
    cls();
    string parentID, complaint;
    if (!inputText("Enter Parent ID (or back): ", parentID)) return;
    if (!inputText("Enter Complaint (or back): ", complaint)) return;

    if (!FileManager::saveComplaint(parentID + "|" + complaint)) {
        cout << "Unable to save complaint.\n";
    } else {
        cout << "\nComplaint Submitted Successfully!\n";
    }
    pauseScreen();
}
void leaveSchoolRequest() {
    cls();
```

```
        string studentID, reason;
        if (!inputText("Enter Student ID (or back): ", studentID)) return;
        if (!inputText("Enter Reason (or back): ", reason)) return;

        if (!FileManager::saveLeaveRequest(studentID + "|" + reason)) {
            cout << "Unable to save request.\n";
        } else {
            cout << "\nLeave Request Sent!\n";
        }
        pauseScreen();
    }
};
class Student : public User {
public:
    bool login(const string& loginID, const string& pass) {
        return FileManager::verifyStudentLogin(loginID, pass);
    }
    void displayMenu() {
        while (true) {
            cls();
            cout << "===== STUDENT PANEL =====\n\n";
            cout << "1. View Profile\n";
            cout << "2. View Attendance\n";
            cout << "3. View Results\n";
            cout << "4. View Fees\n";
            cout << "5. View Notifications\n";
            cout << "6. Logout\n";
            int choice;
            if (!menuChoice("\nEnter Choice: ", choice, 1, 6)) return;
            switch (choice) {
                case 1: viewProfile(); break;
```

```
            case 2: viewAttendance(); break;
            case 3: viewResults(); break;
            case 4: viewFees(); break;
            case 5: viewNotifications(); break;
            case 6: return;
          }
        }
    }
private:
    void viewProfile() {
        cls();
        string id;
        if (!inputText("Enter Student ID (or back): ", id)) return;
        cout << "\n===== PROFILE =====\n";
        FileManager::showStudentProfile(id);
        pauseScreen();
    }
    void viewAttendance() {
        cls();
        cout << "===== ATTENDANCE =====\n\n";
        FileManager::loadAttendance();
        pauseScreen();
    }
    void viewResults() {
        cls();
        cout << "===== RESULTS =====\n\n";
        FileManager::loadResults();
        pauseScreen();
    }
    void viewFees() {
        cls();
```

```
        cout << "===== FEES =====\n\n";
        FileManager::loadFees();
        pauseScreen();
    }
    void viewNotifications() {
        cls();
        cout << "===== NOTIFICATIONS =====\n\n";
        FileManager::loadNotifications();
        pauseScreen();
    }
};
class Teacher : public User {
    string schoolID;
    string subject;
public:
    bool login(const string& loginID, const string& pass) {
    if (!FileManager::verifyTeacherLogin(loginID, pass))
        return false;
    id = loginID;
    schoolID = FileManager::getTeacherSchoolID(loginID);
    return true;
}
    void displayMenu() {
        schoolID = FileManager::getTeacherSchoolID(id);
        while (true) {
            cls();
            cout << "===== TEACHER PANEL =====\n";
            cout << "School ID: " << schoolID << " (" << FileManager::getSchoolName(schoolID) <<
")\n\n";
            cout << "1. Admit Student\n";
            cout << "2. View Students\n";
```

```
            cout << "3. Mark Attendance\n";
            cout << "4. Enter Result\n";
            cout << "5. Set Fees\n";
            cout << "6. Search Student\n";
            cout << "7. Update Result\n";
            cout << "8. Request Student Removal\n";
            cout << "9. Send Notification\n";
            cout << "10. Logout\n";
            int choice;
            if (!menuChoice("\nEnter Choice: ", choice, 1, 10)) return;
            switch (choice) {
                case 1: admitStudent(); break;
                case 2: viewStudents(); break;
                case 3: markAttendance(); break;
                case 4: enterResult(); break;
                case 5: setFees(); break;
                case 6: searchStudent(); break;
                case 7: updateResult(); break;
                case 8: requestStudentRemoval(); break;
                case 9: sendNotification(); break;
                case 10: return;
            }
        }
    }
private:
    void admitStudent() {
        cls();
        cout << "===== ADMIT STUDENT =====\n";
        cout << "School: " << FileManager::getSchoolName(schoolID)
             << " [" << schoolID << "]\n\n";
        string studentID = FileManager::nextID("students.txt", "ST");
```

```
string password, name, dob, aadhar, phone, parentName;
int classNo, rollNo;
char section;
cout << "Generated Student ID: " << studentID << "\n\n";
if (!inputText("Enter Password (or back): ", password)) return;
if (!inputText("Enter Student Name (or back): ", name)) return;
if (!inputText("Enter DOB (or back): ", dob)) return;
if (!inputText("Enter Aadhar (or back): ", aadhar)) return;
if (!inputText("Enter Phone (or back): ", phone)) return;
if (!inputInt("Enter Class (or back): ", classNo, 1, 12)) return;
if (!inputChar("Enter Section (A/B/C... or back): ", section)) return;
section = static_cast<char>(toupper(static_cast<unsigned char>(section)));
if (!inputInt("Enter Roll Number (or back): ", rollNo, 1)) return;
if (!inputText("Enter Parent Name (or back): ", parentName)) return;
string studentData = studentID + "|" + password + "|" + name + "|" + dob + "|" +
                     aadhar + "|" + phone + "|" + schoolID + "|" +
                     to_string(classNo) + "|" + string(1, section) + "|" +
                     to_string(rollNo) + "|" + parentName;
if (!FileManager::saveStudent(studentData)) {
   cout << "\nUnable to save student.\n";
   pauseScreen();
   return;
}
string parentID = FileManager::nextID("parents.txt", "PA");
string parentData = parentID + "|" + password + "|" + parentName + "|" + studentID;
FileManager::saveParent(parentData);
cout << "\nStudent Admitted Successfully!\n";
cout << "Student ID : " << studentID << '\n';
cout << "Parent ID  : " << parentID << '\n';
cout << "School ID  : " << schoolID << '\n';
pauseScreen();
```

```
    }
    void viewStudents() {
        cls();
        cout << "===== STUDENTS OF " << FileManager::getSchoolName(schoolID) << "
=====\n\n";
        vector<string> students = FileManager::readAll("students.txt");
        bool found = false;
        for (const string& line : students) {
            vector<string> f = FileManager::split(line);
            if (f.size() > 6 && f[6] == schoolID) {
                cout << line << '\n';
                found = true;
            }
        }
        if (!found) cout << "No students found.\n";
        pauseScreen();
    }
    bool validateStudentForThisSchool(const string& studentID) {
        if (!FileManager::studentExists(studentID)) {
            cout << "Student Not Found!\n";
            return false;
        }
        if (!FileManager::studentBelongsToSchool(studentID, schoolID)) {
            cout << "Invalid student: this student does not belong to your school.\n";
            return false;
        }
        return true;
    }
    void markAttendance() {
        cls();
        string studentID, date;
```

```
    char status;
    if (!inputText("Enter Student ID (or back): ", studentID)) return;
    if (!validateStudentForThisSchool(studentID)) { pauseScreen(); return; }
    if (!inputText("Enter Date (DD-MM-YYYY or back): ", date)) return;
    if (!inputChar("Enter Status (P/A or back): ", status)) return;
    status = static_cast<char>(toupper(static_cast<unsigned char>(status)));
    if (status != 'P' && status != 'A') {
        cout << "Invalid option\n";
        pauseScreen();
        return;
    }
    if (FileManager::saveAttendance(studentID + "|" + date + "|" + string(1, status)))
        cout << "\nAttendance Marked Successfully!\n";
    else cout << "Unable to save attendance.\n";
    pauseScreen();
}
void enterResult() {
    cls();
    string studentID, subject;
    int marks;
    if (!inputText("Enter Student ID (or back): ", studentID)) return;
    if (!validateStudentForThisSchool(studentID)) { pauseScreen(); return; }
    if (!inputText("Enter Subject (or back): ", subject)) return;
    if (!inputInt("Enter Marks (0-100 or back): ", marks, 0, 100)) return;
    if (FileManager::saveResult(studentID + "|" + subject + "|" + to_string(marks)))
        cout << "\nResult Saved Successfully!\n";
    else cout << "Unable to save result.\n";
    pauseScreen();
}
void setFees() {
    cls();
```

```
    string studentID, status;
    int amount;
    if (!inputText("Enter Student ID (or back): ", studentID)) return;
    if (!validateStudentForThisSchool(studentID)) { pauseScreen(); return; }
    if (!inputInt("Enter Fees Amount (or back): ", amount, 0)) return;
    if (!inputText("Enter Status (Paid/Unpaid or back): ", status)) return;
    if (FileManager::saveFees(studentID + "|" + to_string(amount) + "|" + status))
        cout << "\nFees Saved Successfully!\n";
    else cout << "Unable to save fees.\n";
    pauseScreen();
}
void searchStudent() {
    cls();
    string id;
    if (!inputText("Enter Student ID (or back): ", id)) return;
    cout << "\n===== SEARCH RESULT =====\n";
    if (!FileManager::searchStudent(id, schoolID))
        cout << "Student Not Found!\n";
    pauseScreen();
}
void updateResult() {
    cls();
    string studentID, subject;
    int marks;
    if (!inputText("Enter Student ID (or back): ", studentID)) return;
    if (!validateStudentForThisSchool(studentID)) { pauseScreen(); return; }
    if (!inputText("Enter Subject (or back): ", subject)) return;
    if (!inputInt("Enter New Marks (0-100 or back): ", marks, 0, 100)) return;
    if (FileManager::updateResult(studentID, subject, marks))
        cout << "\nResult Updated Successfully!\n";
    else cout << "\nResult record not found!\n";
```

```
        pauseScreen();
    }
    void requestStudentRemoval() {
        cls();
        string studentID, reason;
        if (!inputText("Enter Student ID (or back): ", studentID)) return;
        if (!validateStudentForThisSchool(studentID)) { pauseScreen(); return; }
        if (!inputText("Enter Reason (or back): ", reason)) return;

        if (FileManager::saveRemovalRequest(studentID + "|" + reason))
            cout << "\nRemoval Request Sent Successfully!\n";
        else cout << "Unable to save request.\n";
        pauseScreen();
    }
    void sendNotification() {
        cls();
        string message;
        if (!inputText("Enter Notification (or back): ", message)) return;
        if (FileManager::saveNotification(id + "|" + schoolID + "|" + message))
            cout << "\nNotification Sent Successfully!\n";
        else cout << "Unable to save notification.\n";
        pauseScreen();
    }
};
class Principal : public User {
    string schoolID;
public:
    bool login(const string& loginID, const string& pass) {
        if (!FileManager::verifyPrincipalLogin(loginID, pass)) return false;
        id = loginID;
        schoolID = FileManager::getPrincipalSchoolID(loginID);
```

```
    return true;
  }
  void displayMenu() {
    while (true) {
      cls();
      cout << "===== PRINCIPAL PANEL =====\n";
      cout << "School ID: " << schoolID << " (" << FileManager::getSchoolName(schoolID) <<
")\n\n";
      cout << "1. Add Teacher\n";
      cout << "2. View Teachers\n";
      cout << "3. Search Teacher\n";
      cout << "4. Remove Teacher\n";
      cout << "5. View Students\n";
      cout << "6. View Attendance\n";
      cout << "7. View Results\n";
      cout << "8. View Complaints\n";
      cout << "9. View Removal Requests\n";
      cout << "10. Send Notification\n";
      cout << "11. Logout\n";
      int choice;
      if (!menuChoice("\nEnter Choice: ", choice, 1, 11)) return;
      switch (choice) {
        case 1: addTeacher(); break;
        case 2: viewTeachers(); break;
        case 3: searchTeacher(); break;
        case 4: removeTeacher(); break;
        case 5: viewStudents(); break;
        case 6: viewAttendance(); break;
        case 7: viewResults(); break;
        case 8: viewComplaints(); break;
        case 9: viewRemovalRequests(); break;
```

```
            case 10: sendNotification(); break;
            case 11: return;
        }
    }
  }
private:
  void addTeacher() {
    cls();
    cout << "===== ADD TEACHER =====\n";
    cout << "School: " << FileManager::getSchoolName(schoolID)
         << " [" << schoolID << "]\n\n";
    string teacherID = FileManager::nextID("teachers.txt", "TE");
    string password, name, dob, aadhar, phone, subject;
    cout << "Generated Teacher ID: " << teacherID << "\n\n";
    if (!inputText("Enter Password (or back): ", password)) return;
    if (!inputText("Enter Name (or back): ", name)) return;
    if (!inputText("Enter DOB (or back): ", dob)) return;
    if (!inputText("Enter Aadhar (or back): ", aadhar)) return;
    if (!inputText("Enter Phone (or back): ", phone)) return;
    if (!inputText("Enter Subject (or back): ", subject)) return;
    string data = teacherID + "|" + password + "|" + name + "|" + dob + "|" +
                  aadhar + "|" + phone + "|" + schoolID + "|" + subject;
    if (FileManager::saveTeacher(data)) {
      cout << "\nTeacher Added Successfully!\n";
      cout << "Teacher ID: " << teacherID << '\n';
      cout << "School ID : " << schoolID << '\n';
    } else {
      cout << "Unable to save teacher.\n";
    }
    pauseScreen();
  }
```

```
    void viewTeachers() {
        cls();
        cout << "===== TEACHERS OF " << FileManager::getSchoolName(schoolID) << "
=====\n\n";
        vector<string> teachers = FileManager::readAll("teachers.txt");
        bool found = false;
        for (const string& line : teachers) {
            vector<string> f = FileManager::split(line);
            if (f.size() > 6 && f[6] == schoolID) {
                cout << line << '\n';
                found = true;
            }
        }
        if (!found) cout << "No teachers found.\n";
        pauseScreen();
    }
    void searchTeacher() {
        cls();
        string id;
        if (!inputText("Enter Teacher ID (or back): ", id)) return;
        cout << "\n===== SEARCH RESULT =====\n";
        if (!FileManager::searchTeacher(id, schoolID))
            cout << "Teacher Not Found!\n";
        pauseScreen();
    }
    void removeTeacher() {
        cls();
        string teacherID;
        if (!inputText("Enter Teacher ID to Remove (or back): ", teacherID)) return;
        if (!FileManager::teacherExists(teacherID)) {
            cout << "Teacher Not Found!\n";
```

```
            pauseScreen();
            return;
        }
        if (FileManager::getTeacherSchoolID(teacherID) != schoolID) {
            cout << "You cannot remove a teacher from another school.\n";
            pauseScreen();
            return;
        }
        if (FileManager::removeByID("teachers.txt", teacherID))
            cout << "\nTeacher Removed Successfully!\n";
        else cout << "Unable to remove teacher.\n";
        pauseScreen();
    }
    void viewStudents() {
        cls();
        cout << "===== STUDENTS OF " << FileManager::getSchoolName(schoolID) << "
=====\n\n";
        vector<string> students = FileManager::readAll("students.txt");
        bool found = false;
        for (const string& line : students) {
            vector<string> f = FileManager::split(line);
            if (f.size() > 6 && f[6] == schoolID) {
                cout << line << '\n';
                found = true;
            }
        }
        if (!found) cout << "No students found.\n";
        pauseScreen();
    }
    void viewAttendance() {
        cls();
```

```
    cout << "===== ATTENDANCE RECORDS =====\n\n";
    FileManager::loadAttendance();
    pauseScreen();
}
void viewResults() {
    cls();
    cout << "===== RESULT RECORDS =====\n\n";
    FileManager::loadResults();
    pauseScreen();
}
void viewComplaints() {
    cls();
    cout << "===== COMPLAINTS =====\n\n";
    FileManager::loadComplaints();
    pauseScreen();
}
void viewRemovalRequests() {
    cls();
    cout << "===== REMOVAL REQUESTS =====\n\n";
    FileManager::loadRemovalRequests();
    pauseScreen();
}
void sendNotification() {
    cls();
    string message;
    if (!inputText("Enter Notification (or back): ", message)) return;
    if (FileManager::saveNotification(id + "|" + schoolID + "|" + message))
        cout << "\nNotification Sent Successfully!\n";
    else cout << "Unable to save notification.\n";
    pauseScreen();
}
```

```
};
class SuperAdmin {
   string password = "malik";
public:
   bool login(const string& enteredPassword) const {
      return enteredPassword == password;
   }
   void displayMenu() {
      while (true) {
         cls();
         cout << "===== SUPER ADMIN PANEL =====\n\n";
         cout << "1. Create School\n";
         cout << "2. View Schools\n";
         cout << "3. Add Principal\n";
         cout << "4. View Principals\n";
         cout << "5. Delete School\n";
         cout << "6. Remove Principal\n";
         cout << "7. Edit School\n";
         cout << "8. Global Notification\n";
         cout << "9. Logout\n";
         int choice;
         if (!menuChoice("\nEnter Choice: ", choice, 1, 9)) return;
         switch (choice) {
            case 1: createSchool(); break;
            case 2: viewSchools(); break;
            case 3: addPrincipal(); break;
            case 4: viewPrincipals(); break;
            case 5: deleteSchool(); break;
            case 6: removePrincipal(); break;
            case 7: editSchool(); break;
            case 8: globalNotification(); break;
```

```
            case 9: return;
        }
    }
  }
private:
  void createSchool() {
    cls();
    cout << "===== CREATE SCHOOL =====\n\n";
    string schoolName;
    int totalClasses, totalSections;
    string schoolID = FileManager::nextID("schools.txt", "SC");
    cout << "Generated School ID: " << schoolID << "\n\n";
    if (!inputText("Enter School Name (or back): ", schoolName)) return;
    if (!inputInt("Enter Total Classes (or back): ", totalClasses, 1)) return;
    if (!inputInt("Enter Total Sections (or back): ", totalSections, 1)) return;
    string data = schoolID + "|" + schoolName + "||" +
                  to_string(totalClasses) + "|" + to_string(totalSections);
    if (FileManager::saveSchool(data)) {
      cout << "\nSchool Created Successfully!\n";
      cout << "School ID: " << schoolID << '\n';
    } else {
      cout << "Unable to save school.\n";
    }
    pauseScreen();
  }
  void viewSchools() {
    cls();
    cout << "===== ALL SCHOOLS =====\n\n";
    FileManager::loadSchools();
    pauseScreen();
  }
```

```
void addPrincipal() {
    cls();
    cout << "===== ADD PRINCIPAL =====\n\n";
    string schoolID;
    if (!inputText("Enter School ID (SC001) or back: ", schoolID)) return;
    if (!FileManager::schoolExists(schoolID)) {
        cout << "School ID not found! No random school names allowed\n";
        pauseScreen();
        return;
    }
    string principalID = FileManager::nextID("principals.txt", "PR");
    string password, name, dob, aadhar, phone;
    cout << "Generated Principal ID: " << principalID << "\n";
    cout << "School: " << FileManager::getSchoolName(schoolID) << " [" << schoolID <<
"]\n\n";
    if (!inputText("Enter Password (or back): ", password)) return;
    if (!inputText("Enter Name (or back): ", name)) return;
    if (!inputText("Enter DOB (or back): ", dob)) return;
    if (!inputText("Enter Aadhar (or back): ", aadhar)) return;
    if (!inputText("Enter Phone (or back): ", phone)) return;
    string data = principalID + "|" + password + "|" + name + "|" + dob + "|" +
            aadhar + "|" + phone + "|" + schoolID;
    if (!FileManager::savePrincipal(data)) {
        cout << "Unable to save principal.\n";
        pauseScreen();
        return;
    }
    string schoolLine;
    if (FileManager::findByID("schools.txt", schoolID, schoolLine)) {
        vector<string> f = FileManager::split(schoolLine);
        if (f.size() >= 5) {
```

```
            f[2] = principalID;
            string updated = f[0];
            for (size_t i = 1; i < f.size(); ++i) updated += "|" + f[i];
            FileManager::updateLine("schools.txt", schoolID, updated);
        }
    }
    cout << "\nPrincipal Added Successfully!\n";
    cout << "Principal ID: " << principalID << '\n';
    cout << "School ID   : " << schoolID << '\n';
    pauseScreen();
}
void viewPrincipals() {
    cls();
    cout << "===== ALL PRINCIPALS =====\n\n";
    FileManager::loadPrincipals();
    pauseScreen();
}
void removePrincipal() {
    cls();
    string principalID;
    if (!inputText("Enter Principal ID to Remove (or back): ", principalID)) return;
    if (!FileManager::principalExists(principalID)) {
        cout << "Principal Not Found!\n";
        pauseScreen();
        return;
    }
    string schoolID = FileManager::getPrincipalSchoolID(principalID);
    if (FileManager::removeByID("principals.txt", principalID)) {
        string schoolLine;
        if (FileManager::findByID("schools.txt", schoolID, schoolLine)) {
            vector<string> f = FileManager::split(schoolLine);
```

```
            if (f.size() >= 5) {
                f[2] = "";
                string updated = f[0];
                for (size_t i = 1; i < f.size(); ++i) updated += "|" + f[i];
                FileManager::updateLine("schools.txt", schoolID, updated);
            }
        }
        cout << "\nPrincipal Removed Successfully!\n";
    } else {
        cout << "Unable to remove principal.\n";
    }
    pauseScreen();
}
void deleteSchool() {
    cls();
    string schoolID;
    if (!inputText("Enter School ID to Delete (or back): ", schoolID)) return;
    if (!FileManager::schoolExists(schoolID)) {
        cout << "School Not Found!\n";
        pauseScreen();
        return;
    }
    if (FileManager::deleteSchool(schoolID))
        cout << "\nSchool Deleted Successfully!\n";
    else
        cout << "Unable to delete school.\n";
    pauseScreen();
}
void editSchool() {
    cls();
    string schoolID, newName;
```

```
    if (!inputText("Enter School ID (or back): ", schoolID)) return;
    if (!FileManager::schoolExists(schoolID)) {
        cout << "School Not Found!\n";
        pauseScreen();
        return;
    }
    if (!inputText("Enter New School Name (or back): ", newName)) return;
    string line;
    if (!FileManager::findByID("schools.txt", schoolID, line)) {
        cout << "School Not Found!\n";
        pauseScreen();
        return;
    }
    vector<string> f = FileManager::split(line);
    if (f.size() < 5) {
        cout << "School record is damaged.\n";
        pauseScreen();
        return;
    }
    f[1] = newName;
    string updated = f[0];
    for (size_t i = 1; i < f.size(); ++i) updated += "|" + f[i];

    if (FileManager::updateLine("schools.txt", schoolID, updated))
        cout << "\nSchool Updated Successfully!\n";
    else
        cout << "Unable to update school.\n";
    pauseScreen();
}
void globalNotification() {
    cls();
```

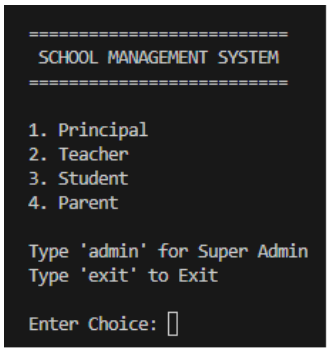
```
        string message;
        if (!inputText("Enter Global Notification (or back): ", message)) return;
        if (FileManager::saveNotification("GLOBAL|" + message))
            cout << "\nGlobal Notification Sent!\n";
        else cout << "Unable to save notification.\n";
        pauseScreen();
    }
};
}
using namespace SMS;
int main() {
    while (true) {
        cls();
        cout << "==========================\n";
        cout << " SCHOOL MANAGEMENT SYSTEM\n";
        cout << "==========================\n\n";
        cout << "1. Principal\n";
        cout << "2. Teacher\n";
        cout << "3. Student\n";
        cout << "4. Parent\n\n";
        cout << "Type 'admin' for Super Admin\n";
        cout << "Type 'exit' to Exit\n";
        cout << "Type 'back' to stay here\n\n";
        cout << "Enter Choice: ";
        string choice;
        getline(cin >> ws, choice);
        string lower = lowerCopy(choice);
        if (lower == "exit") {
            cls();
            cout << "Thank You For Using SMS!\n";
            break;
```

```
    }
    if (lower == "back") continue;
    if (lower == "admin") {
        cls();
        SuperAdmin admin;
        string password;
        if (!inputText("Enter Admin Password (or back): ", password)) continue;
        if (admin.login(password)) {
            cout << "\nLogin Successful!\n";
            pauseScreen();
            admin.displayMenu();
        } else {
            cout << "\nWrong Password!\n";
            pauseScreen();
        }
    }else if (choice == "1") {
        cls();
        Principal p;
        string id, password;
        if (!inputText("Enter Principal ID (or back): ", id)) continue;
        if (!inputText("Enter Password (or back): ", password)) continue;
        if (p.login(id, password)) {
            cout << "\nLogin Successful!\n";
            pauseScreen();
            p.displayMenu();
        } else {
            cout << "\nInvalid Credentials!\n";
            pauseScreen();
        }
    }else if (choice == "2") {
        cls();
```

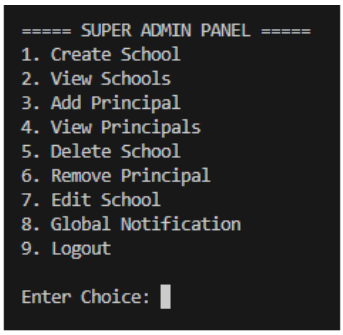
```
    Teacher t;
    string id, password;
    if (!inputText("Enter Teacher ID (or back): ", id)) continue;
    if (!inputText("Enter Password (or back): ", password)) continue;
    if (t.login(id, password)) {
        cout << "\nLogin Successful!\n";
        pauseScreen();
        t.displayMenu();
    } else {
        cout << "\nInvalid Credentials!\n";
        pauseScreen();
    }
}else if (choice == "3") {
    cls();
    Student s;
    string id, password;
    if (!inputText("Enter Student ID (or back): ", id)) continue;
    if (!inputText("Enter Password (or back): ", password)) continue;
    if (s.login(id, password)) {
        cout << "\nLogin Successful!\n";
        pauseScreen();
        s.displayMenu();
    } else {
        cout << "\nInvalid Credentials!\n";
        pauseScreen();
    }
}else if (choice == "4") {
    cls();
    Parent p;
    string id, password;
    if (!inputText("Enter Parent ID (or back): ", id)) continue;
```

```
            if (!inputText("Enter Password (or back): ", password)) continue;
            if (p.login(id, password)) {
                cout << "\nLogin Successful!\n";
                pauseScreen();
                p.displayMenu();
            } else {
                cout << "\nInvalid Credentials!\n";
                pauseScreen();
            }
        }else{
            invalidChoice();
        }
    }
    return 0;
}
```

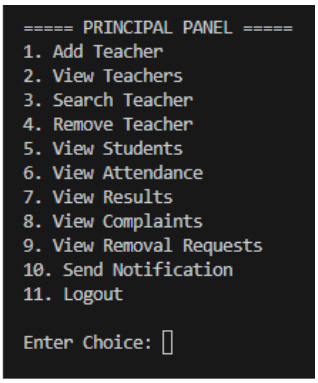
4.2 Output:



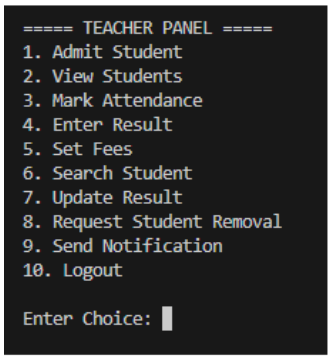
Main menu



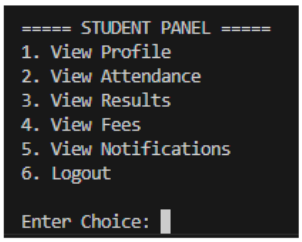
Super admin menu



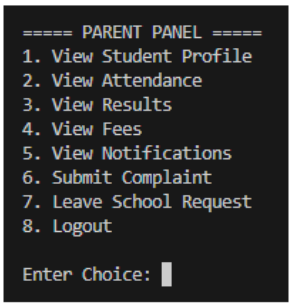
Principal menu



Teacher menu



Student menu



Parent menu

CHAPTER 5: CONCLUSION

5.1 Future Scope of the Project:

- ❖ Addition of a graphical user interface (GUI).
- ❖ Online access for students, parents and teachers.
- ❖ Integration with a database for better data management.
- ❖ Addition of online fee payment and report generation.
- ❖ Development of a mobile or web-based version.
- ❖ More features can be added according to future requirements.

5.2 Limitations:

- The current system is console-based.
- Data is stored using local files.
- The system does not provide online or remote access.
- Requires manual entry of student and staff information.
- Limited security compared to a database-based system.

5.3 Security:

- Provides separate login access for different users.
- Users can access functions according to their roles.
- Important operations are restricted to authorized users.
- Records are stored locally through file handling.
- Helps prevent unauthorized modification of school records.

5.4 Conclusion:

- ❖ The **School Management System** successfully demonstrates the use of C++ for managing different school activities. It helps organize records of schools, principals, teachers, students, attendance and results while reducing manual work.

The project provides a simple, organized and efficient system using **C++ programming and file handling**.

CHAPTER 6: BIBLIOGRAPHY

6.1 Books:

- **Sumita Arora**, *Computer Science with C++*.
- **E. Balagurusamy**, *Object-Oriented Programming with C++*.
- **Bjarne Stroustrup**, *The C++ Programming Language*.